Ho Ting (Bosco) Hung[1]

January 17, 2026

Westerlund: Westerlund ECM Panel Cointegration Test

Abstract: This paper introduces a new Python and R package: **Westerlund**. The Python and R codes implement the Westerlund (2007) Panel Cointegration Tests, which are used to determine if there is a long-run relationship (cointegration) between a dependent variable y and a set of independent variables x across a panel of N cross-sectional units over T time periods. The code provides a functional approximation of the Stata command `xtwest` developed by Persyn and Westerlund (2008) and also offers an intuitive visualization diagnostic tool to facilitate the examination of the bootstrap distribution and significance of the statistics of interest.

[1]Department of Politics and International Relations, University of Oxford
bosco.hung@st-annes.ox.ac.uk

Contents

1 Overview

Econometric models, such as the Autoregressive Distributed Lag (ARDL) models, are increasingly used to analyze the long-run equilibrium relationship between a dependent variable y and a set of independent variables x across a panel of N cross-sectional units over T time periods (Pesaran & Shin, 1999; Pesaran et al., 2001). Before applying such models to the panel data, one has to first determine if a long-run equilibrium relationship exists between the variables using a cointegration test. However, panel data often suffer from the problem of cross-sectional dependence (CD/CSD), i.e. the units of observations are not independent of one another, and this is typically caused by unobserved common factors that affect all units (e.g. global financial crisis in the case of investment flows) (Pesaran, 2021). Traditional residuals-based tests like the Pedroni panel cointegration test enforce strict exogeneity constraints and are not robust to cross-sectional dependence, so they are vulnerable to over-rejection (Pedroni, 2004). In cases of cross-sectional dependence, tests based on structural dynamics like the Westerlund (2007) test have to be used, which is now only available on Stata (Persyn & Westerlund, 2008).

This paper introduces a Python and R functional approximation implementation of the Westerlund (2007) tests, **Westerlund**, which can be accessed at https://github.com/bosco-hung/WesterlundTest. The Python version is also available at PyPI (https://pypi.org/project/Westerlund/) and the R version is currently under review. The implementations provide similar functionalities of the Stata command `xtwest` developed by Persyn and Westerlund (2008), including the calculation of the four primary test statistics (G_τ, G_α, P_τ, P_α) through asymptotic standardization using moments from Westerlund (2007), and a bootstrap procedure to handle cross-sectional dependence. An additional bootstrapping visualization function is added to facilitate the examination of the bootstrap distribution and significance of the statistics of interest.

This paper starts by discussing the econometric model developed by Westerlund (2007). Then, it discusses the Python and R implementations. It proceeds with some points to note and closes with some implementation examples.

2 The Econometric Model

The Westerlund test is based on a structural equation known as the Conditional Error Correction Model (ECM):

$$\Delta y_{it} = \underbrace{\delta_i' d_t}_{\text{Deterministics}} + \underbrace{\alpha_i (y_{i,t-1} - \beta_i' x_{i,t-1})}_{\text{Long-Run Equilibrium}} + \underbrace{\sum_{j=1}^{p_i} \alpha_{ij} \Delta y_{i,t-j} + \sum_{j=-q_i}^{p_i} \gamma_{ij} \Delta x_{i,t-j}}_{\text{Short-Run Dynamics}} + e_{it} \tag{1}$$

where its core lies in the long-run equilibrium term $\alpha_i(y_{i,t-1} - \beta_i' x_{i,t-1})$.[2] The expression $y_{i,t-1} - \beta_i' x_{i,t-1}$ represents the deviation from equilibrium in the previous time period. If variables y and x are cointegrated, they move together in the long run according to the relationship $y_{it} = \beta_i x_{it}$. Therefore, the residual of $y_{i,t-1} - \beta_i' x_{i,t-1}$ represents the "error" or "disequilibrium" at time $t-1$. The parameter α_i determines how the system responds to that disequilibrium. If $\alpha_i < 0$ (Error Correction), the system is stable. If y_{t-1} was "too high" relative to x_{t-1} (positive error), the negative α_i forces Δy_{it} to be negative. The variable y falls to restore equilibrium. On the other hand, if $\alpha_i = 0$ (No Error Correction), the change in y (Δy_{it}) does not depend on the previous level. The variables drift apart indiscriminately. This implies no cointegration.

The summation terms $\sum_{j=1}^{p_i} \phi_{ij} \Delta y_{i,t-j} + \sum_{j=-q_i}^{p_i} \gamma_{ij} \Delta x_{i,t-j}$ represent the short-run shocks. These terms $\sum_{j=1}^{p_i} \phi_{ij} \Delta y_{i,t-j}$ capture the inertia in the system. By including sufficient lags of Δy and Δx, we ensure that the regression residual e_{it} is free of serial correlation (white noise), so as to ensure the validity of the OLS t-statistics. On the other hand, the term $\sum_{j=-q_i}^{0} \gamma_{ij} \Delta x_{i,t-j}$ includes current and future differences of x for correcting for endogeneity. By projecting the error onto the leads and lags of Δx, the regressors become strictly exogenous with respect to the error term, allowing for valid asymptotic inference on α_i.

The term $\delta_i' d_t$ accounts for trends in the data that are not related to the stochastic relationship between x and y. In Case 1 ($d_t = 0$), there is no intercept. The data has a zero mean; In Case 2 ($d_t = \{1\}$), there is a constant intercept. The data fluctuates around a non-zero level; Finally, in Case 3 ($d_t = \{1, t\}$), there is both an intercept and linear trend. The data drifts upward or downward over time.

2.1 Hypothesis Testing Framework

The statistical test tests the value of α_i derived from the equation outlined in the following. Under the Null Hypothesis (H_0), there is no error correction, i.e. there is no cointegration exists for any unit:

$$H_0 : \alpha_i = 0 \quad \text{for all } i = 1 \dots N \tag{2}$$

[2] $-\alpha_i \beta_i'$ is sometimes written as λ_i'.

The Alternative Hypothesis (H_1) depends on which statistic is used (Group vs. Panel): In the case of group mean statistics (G_τ, G_α), they do not assume a common speed of adjustment. They test whether cointegration exists for at least one unit in the panel:

$$H_1^G : \alpha_i < 0 \quad \text{for at least one } i \tag{3}$$

In the case of panel mean statistics (P_τ, P_α), they "pool" the information across the cross-sectional dimension and test whether cointegration exists for the entire panel as a whole, assuming a homogeneous speed of adjustment:

$$H_1^P : \alpha_i = \alpha < 0 \quad \text{for all } i \tag{4}$$

3 R

The R version of the code features a main driver: `westerlund_test`. This primary interface connects the functions to perform input validation and data sorting, as well as to handle the branching logic between asymptotic and bootstrap inference, which are summarized in Figure 1. It takes the following arguments:

```
westerlund_test(data,
                yvar,
                xvars,
                idvar,
                timevar,
                constant = FALSE,
                trend = FALSE,
                lags,              # Can be a single integer or a vector
                leads = NULL,      # Can be a single integer or a vector
                westerlund = FALSE,
                aic = TRUE,
                indiv.ecm = FALSE,
                bootstrap = -1,    # Default -1
                lrwindow = 2,
                verbose = FALSE)
```

where `data` is the panel data input (ideally balanced panel), `yvar` is a string specifying the name of the dependent variable, and `xvars` is a character vector of regressors entering the long-run relationship, allowing a maximum of six variables or a single variable if `westerlund=TRUE`. The `idvar` string identifies cross-sectional units, while `timevar` identifies the time column. Inclusion of an intercept or a linear trend is controlled by the `constant` and `trend` logical arguments, where setting `trend=TRUE` necessitates that `constant` also be `TRUE`. The short-run lag length p is defined by `lags`, and the lead length q is defined by `leads` (defaulting to 0 if `NULL`); both accept a single integer or a range provided as a vector, where the latter will enable auto selection using either AIC or BIC as the information criterion controlled by the `aic` parameter. If the `westerlund` flag is `TRUE`, the function enforces additional restrictions requiring at least a constant and no more than one regressor. The `indiv.ecm` parameter manages whether individual regression outputs will be obtained. Finally, `bootstrap` determines the number of replications used to calculate bootstrap p-values for the statistics, `lrwindow` sets the window parameter for long-run variance estimation, which defaults to 2, and `verbose` toggles the display of detailed information.

The implementation enforces strict time-series continuity by checking for time gaps:

$$\Delta t = t_{i,s} - t_{i,s-1} \tag{5}$$

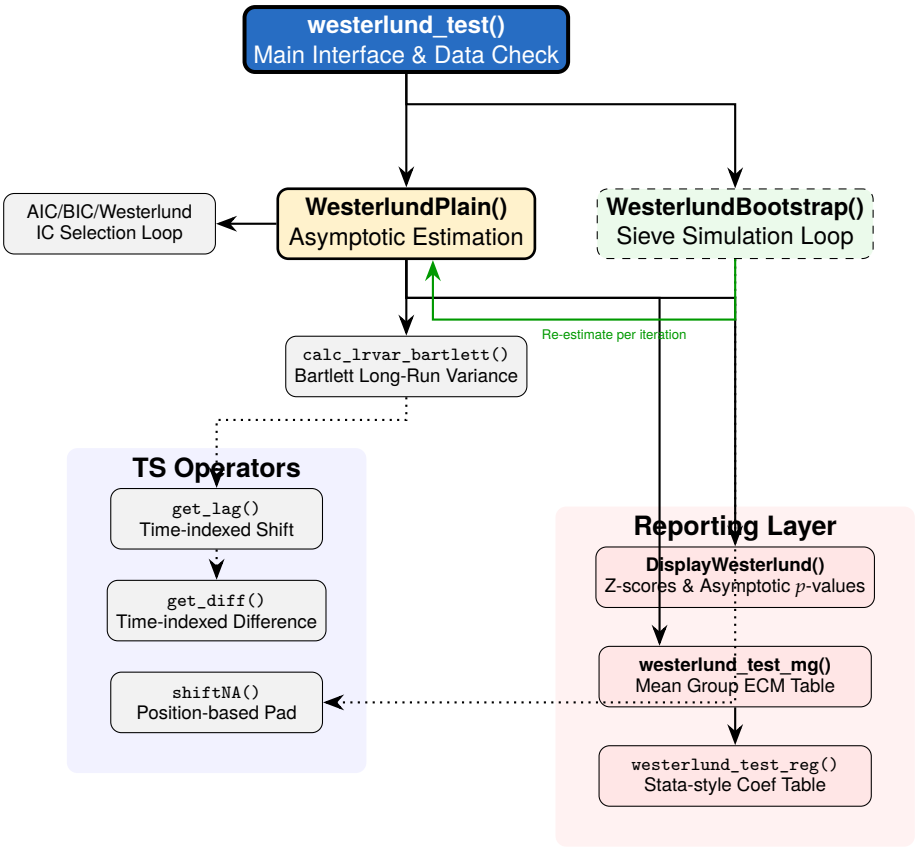

Figure 1: *Structure of the R Version.*

If $\Delta t > 1$, a "hole" is identified, and the function halts to prevent invalid differencing. The data is sorted to ensure the vector Y follows a block-unit structure:

$$Y = [y_{1,1}, y_{1,2}, \cdots, y_{1,T}, \quad y_{2,1}, \cdots, y_{2,T}, \quad \cdots, \quad y_{N,T}]' \tag{6}$$

However, it is expected that the data input will be a balanced panel, as other preceding or subsequent parts of the typical estimation workflow will demand or prefer a balanced panel. Moreover, as the Westerlund test involves a high density of parameters, a balanced panel is preferred to preserve as many observations as possible, especially in the case of a small panel (e.g. macroeconomics panel with annual data). The workflow integration will be discussed later in Section 3.5.

3.1 Econometric Estimation Logic

The `WesterlundPlain` function estimates ECMs and aggregates them into panel statistics.

3.1.1 Unit-Level Estimation

For each unit i, the code estimates the unrestricted ECM outlined in Equation 1. If `auto` selection is enabled, the code minimizes the Akaike information criterion (AIC) or the Bayesian information criterion (BIC). If `westerlund=TRUE`, it uses the specific penalty:

$$AIC(p,q) = \ln\left(\frac{RSS_i}{T_i - p - q - 1}\right) + \frac{2(p+q+\det+1)}{T_i - p_{max} - q_{max}} \tag{7}$$

3.1.2 Long-Run Variance (HAC) Estimation

The helper `calc_lrvar_bartlett` computes the Newey-West variance using a Bartlett kernel (Newey & West, 1994). Following the Stata convention, autocovariances $\hat{\gamma}_j$ are scaled by $1/n$:

$$\hat{\omega}^2 = \hat{\gamma}_0 + 2\sum_{j=1}^{M}\left(1 - \frac{j}{M+1}\right)\hat{\gamma}_j, \quad \hat{\gamma}_j = \frac{1}{n}\sum_{t=j+1}^{n}\hat{e}_t\hat{e}_{t-j} \tag{8}$$

3.1.3 Panel Statistics Construction

3.1.3.1 Group Mean Statistics (G) The group mean statistics (G) are computed by averaging individual unit results. Following the completion of the unit-level ECMs, the code aggregates the individual speed-of-adjustment coefficients ($\hat{\alpha}_i$) and their standard errors:

$$G_\tau = \frac{1}{N}\sum_{i=1}^{N}\frac{\hat{\alpha}_i}{SE(\hat{\alpha}_i)} \quad \text{and} \quad G_\alpha = \frac{1}{N}\sum_{i=1}^{N}\frac{T_i\hat{\alpha}_i}{\hat{\alpha}_i(1)} \tag{9}$$

where $\hat{\alpha}_i(1) = \frac{\hat{\omega}_{ui}}{\hat{\omega}_{yi}}$ is the ratio of long-run standard deviations derived from the Bartlett-kernel HAC estimation described earlier.

3.1.3.2 Pooled Panel Statistics (P) For pooled statistics (P_τ, P_α), the code partials out short-run dynamics and deterministics from Δy_{it} and $y_{i,t-1}$ using the average lag and lead orders calculated across all units. The pooled coefficient $\hat{\alpha}_{pooled}$ is then estimated by aggregating these filtered residuals:

$$\hat{\alpha}_{pooled} = \left(\sum_{i=1}^{N}\sum_{t=1}^{T}\tilde{y}_{i,t-1}^2\right)^{-1}\sum_{i=1}^{N}\sum_{t=1}^{T}\frac{1}{\hat{\alpha}_i(1)}\tilde{y}_{i,t-1}\Delta\tilde{y}_{it} \tag{10}$$

The final statistics are then constructed as:

$$P_\tau = \frac{\hat{\alpha}_{pooled}}{SE(\hat{\alpha}_{pooled})} \quad \text{and} \quad P_\alpha = T \cdot \hat{\alpha}_{pooled} \tag{11}$$

where T represents the effective sample size after accounting for the loss of observations due to the chosen lag and lead lengths.

3.2 Standardization and Display

The function `DisplayWesterlund` converts raw statistics S into Z-scores using asymptotic moments μ_S and σ_S:

$$Z_S = \frac{\sqrt{N}(S - \mu_S)}{\sigma_S} \tag{12}$$

The moments are retrieved from hard-coded matrices indexed by deterministic cases (1: None, 2: Constant, 3: Trend) and the number of regressors K in the original Westerlund (2007) paper.

As the test is lower-tailed, the null of no cointegration is rejected if the observed statistic is significantly negative.

3.3 Bootstrap Inference

As explained earlier, the Westerlund test allows the handling of cross-sectional dependence through a bootstrapping logic. To do so, the code implements a recursive sieve bootstrap under the null hypothesis ($H_0 : \alpha_i = 0$) following a multi-stage process:

1. **Null-Model Estimation:** For each unit i, the model is estimated under H_0. Residuals ($\hat{e}_{it}$) and regressor differences (Δx_{it}) are extracted and centered within each unit to ensure the bootstrap innovations have a zero mean. Centering is performed within each unit, whereas the mean of Δx_{it} is calculated only over the "residual support" (rows where $\hat{e}_{it}$ is non-missing) to ensure the bootstrap innovations are consistent with the estimation sample.

2. **Synchronized Temporal Shuffle:** To preserve the contemporaneous correlation structure $E[e_{it}e_{jt}] \neq 0$, the code implements a synchronized shuffling mechanism. First, it identifies U, the minimum number of valid residual observations across all units. A vector of U indices is drawn with replacement and duplicated (expanding the pool to $2U$). A single vector of random draws $U \sim (0, 1)$ is generated as a global shuffle key. A stable sort (using the original index as a tie-breaker) is applied to this key to create a master permutation. Every cross-sectional unit then draws its time-indices from the first T_{ext} elements of this master permutation, so as to ensure that the cross-sectional "link" between units i and j at time t remains intact in the simulated data.

3. **Innovation Construction (u^*_{it}):** The bootstrap innovation is constructed by combining the reshuffled residuals e^*_{it} with the lags and leads of the centered regressor differences:

$$u^*_{it} = e^*_{it} + \sum_{j=-q}^{p} \hat{\gamma}_{ij} \Delta x^*_{i,t-j} \tag{13}$$

The code utilizes a `shiftNA` helper function which initializes an `NA` vector of length n and only populates the shifted indices if $n > |k|$. This ensures that any "out-of-bounds" shifts or boundary gaps result in `NA` values. These `NA`s propagate through the summation, so as to ensure the innovation u^*_{it} is only valid where all components exist. These values are converted to zeros immediately before the recursive step to provide a clean initialization for the AR process.

4. **Recursive Simulation:** The series of first-differences Δy^*_{it} is generated recursively using the unit-specific autoregressive coefficients $\hat{\phi}_{ij}$:

$$\Delta y^*_{it} = \sum_{j=1}^{p} \hat{\phi}_{ij} \Delta y^*_{i,t-j} + u^*_{it} \tag{14}$$

The recursion is performed for the full length of the reshuffled indices. After the loop, the first p periods (the burn-in period) are explicitly set to `NA` to replicate the finite-sample behavior and degrees-of-freedom loss of the original test.

5. **Integration:** The simulated differences are integrated to recover the level variables for both y and x using the `cumsum` function:

$$y^*_{it} = \sum_{s=1}^{t} \Delta y^*_{is}, \quad X^*_{it} = \sum_{s=1}^{t} \Delta X^*_{is} \tag{15}$$

Rows containing `NA` values (due to lags or padding) are dropped before the final panel is re-estimated using the `WesterlundPlain` routine.

Finally, the Robust p-value is computed as:

$$p^* = \frac{\sum_{b=1}^{B} I(Stat^*_b \leq Stat_{obs}) + 1}{B + 1} \tag{16}$$

where finite sample corrections are applied.

3.4 Visualization

The R implementation provides the `plot_westerlund_bootstrap` function to visualize the results of the bootstrap procedure. It leverages the **ggplot2** framework to create a faceted 2 × 2 grid for a side-by-

side comparison of the group mean (G_τ, G_α) and panel (P_τ, P_α) statistics. This allows the researcher to observe if cointegration is supported by individual units or the panel as a whole.

Mathematically, given B bootstrap replications, the function estimates the empirical null distribution using a kernel density estimator:

$$\hat{f}(s) = \frac{1}{Bh} \sum_{b=1}^{B} K\left(\frac{s - \text{Stat}_b^*}{h}\right) \tag{17}$$

where $Stat_b^*$ represents the simulated statistics for $b = 1, \ldots, B$, $K(\cdot)$ is the Gaussian kernel, and h is the bandwidth automatically determined by the `geom_density` geometry.

The visualization logic is structured as follows. First, the area under the kernel density curve is shaded to represent the probability mass of the null hypothesis of no cointegration. Second, a solid vertical line (defaulting to orange) marks the calculated test statistic ($Stat_{obs}$) from the original sample. Third, a dashed vertical line (defaulting to blue) indicates the empirical α-level critical value (CV^*), calculated as the α-quantile of the bootstrap distribution. Fourth, each facet includes text labels for the exact values of $Stat_{obs}$ and CV^* for precise interpretation.

The null hypothesis is rejected at the α level if the observed statistic (solid line) lies to the *left* of the bootstrap critical value (dashed line). If the density mass is located significantly to the right of the observed statistic, the result is considered robust. Significant overlap between the density and the observed statistic suggests that asymptotic significance may be a false positive caused by cross-sectional dependence.

Figure 2 provides an example of the visualization result generated by replicating the first table in p.240 in Persyn and Westerlund (2008).

3.5 Integration with the Other Packages

The **Westerlund** package can be easily integrated into the workflow of the users investigating the long-run relationship of variables in panel data. Examinations of long-run dynamics often involve (1) testing the existence of cross-sectional dependence, (2) testing the order of integration, (3) testing the existence of cointegration, and finally (4) running the error-correction model tests. Regarding cross-sectional dependence, in the R ecosystem, the Pesaran cross-sectional dependence test is available (Pesaran, 2021) in `pcdtest` of the **plm** package (Croissant et al., 2006). Regarding the order of integration, the **plm** package provides several first-generation tests, including the Levin-Lin-Chu (LLC) (Levin et al., 2002), Im-Pesaran-Shin (IPS) (Im et al., 2003), and Hadri stationarity tests (Hadri, 2000). In case there is cross-sectional dependence, the **plm** package also offers the second-generation Cross-sectionally Augmented IPS (CIPS) test proposed by Pesaran (2007) which is robust to cross-sectional dependence. Finally, researchers can use **PooledMeanGroup**, **ARDL**, or **pmg** to run mean group (MG) or panel mean group

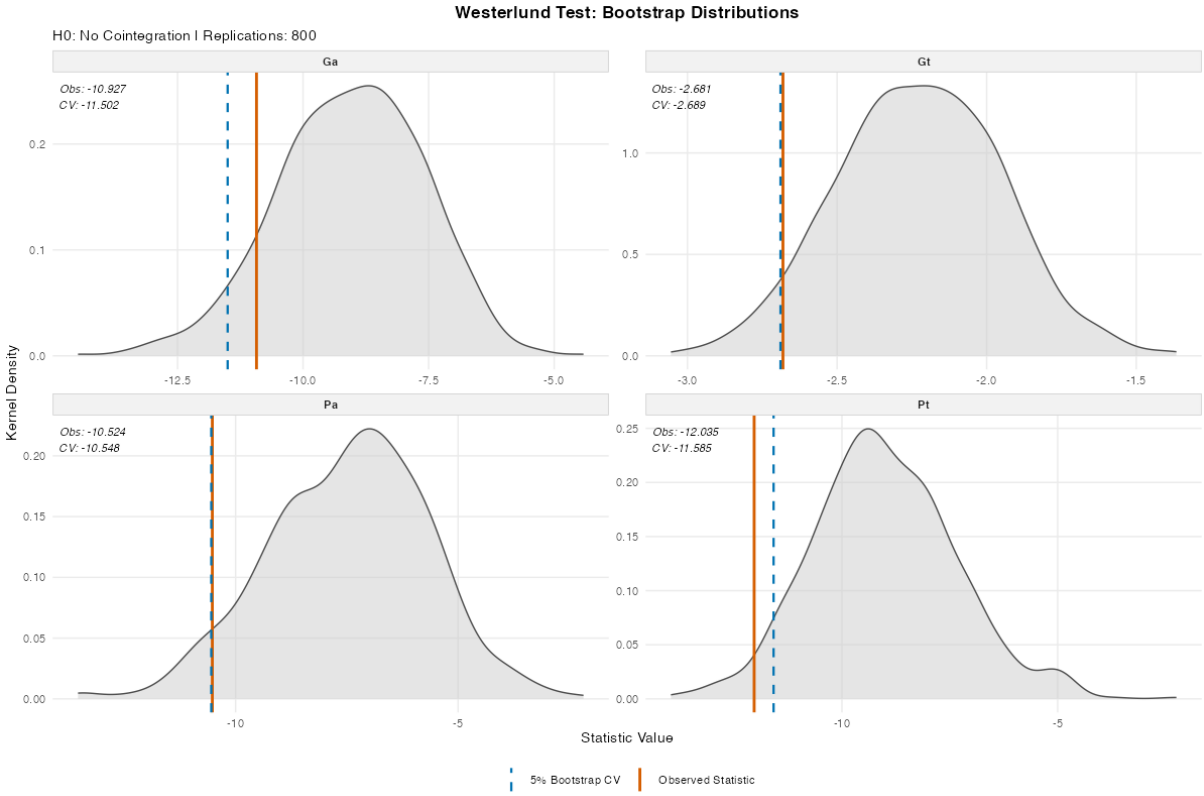

Figure 2: *Bootstrapping Distribution: R Replication of the First Table in p.240 in Persyn and Westerlund (2008).*

(PMG) ARDL models (Natsiopoulos & Tzeremes, 2020; Zientara & Kujawski, 2017). The **Westerlund** package introduced by this paper solves the road block regarding the test of the existence of cointegration, as summarized in Figure 3.

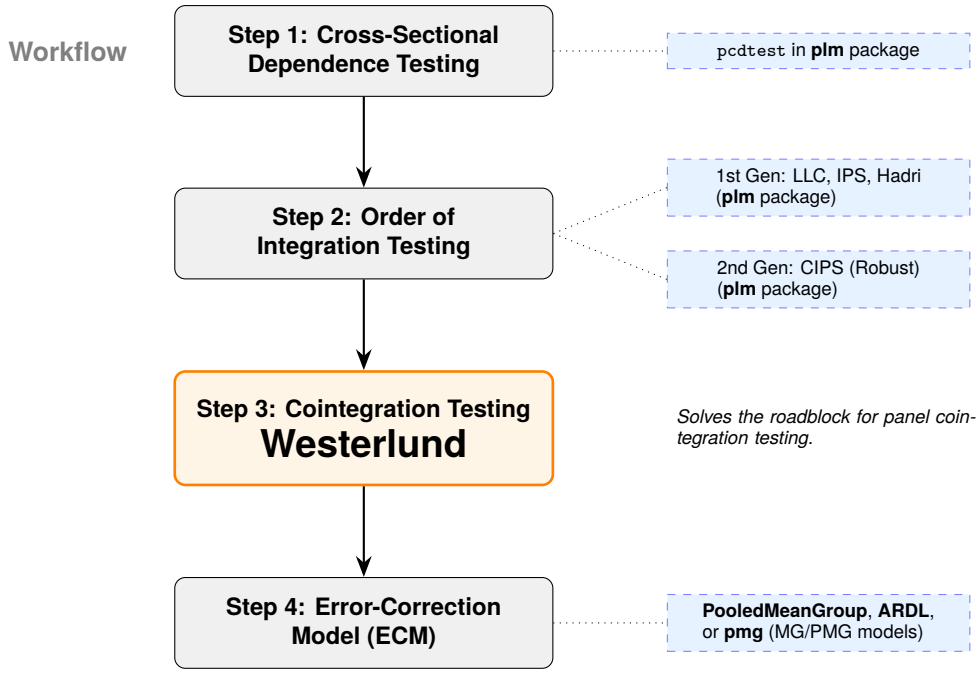

Figure 3: *Workflow Integration.*

4 Python

A Python version of the Westerlund Test framework is also developed to accommodate the preferences of different users. The `WesterlundTest` class provides a modular framework for evaluating cointegration in panel data and its structure is summarized in Figure 4. The class can be initialized as follows:

```
WesterlundTest(self, data, y_var, x_vars, id_var, time_var,
        lags=1, leads=0, lrwindow=2,  constant=False, trend=False,
        westerlund=False, aic=True, indiv_ecm=False, bootstrap=-1,
        seed=None, verbose=False)
```

The `data` parameter expects a `pandas.DataFrame` containing the variables for the panel cointegration test, where `y_var` specifies the dependent variable and `x_vars` identifies the regressor or list of regressors entering the long-run relationship. The cross-sectional units and time index are defined by `id_var` and `time_var` respectively, while the short-run dynamics are controlled by `lags` and `leads`, both of which accept either a fixed integer or a tuple for automatic range selection. The Bartlett kernel bandwidth for long-run variance estimation is set via `lrwindow`, and the deterministic components are managed by `constant` and `trend`, the latter of which automatically forces the inclusion of an intercept. If the `westerlund` parameter is enabled, the model enforces specific constraints such as specialized information criteria and trimming logic. Otherwise, the `aic` parameter controls whether the AIC or BIC is used as the information criteria when auto selection is enabled. The `indiv_ecm` manages whether unit-specific regression results are produced as an output. `bootstrap` specifies the number of replications for robust

p-value calculation, using an optional `seed` for reproducibility. Finally, the `verbose` argument determines whether detailed messages are printed to the console.

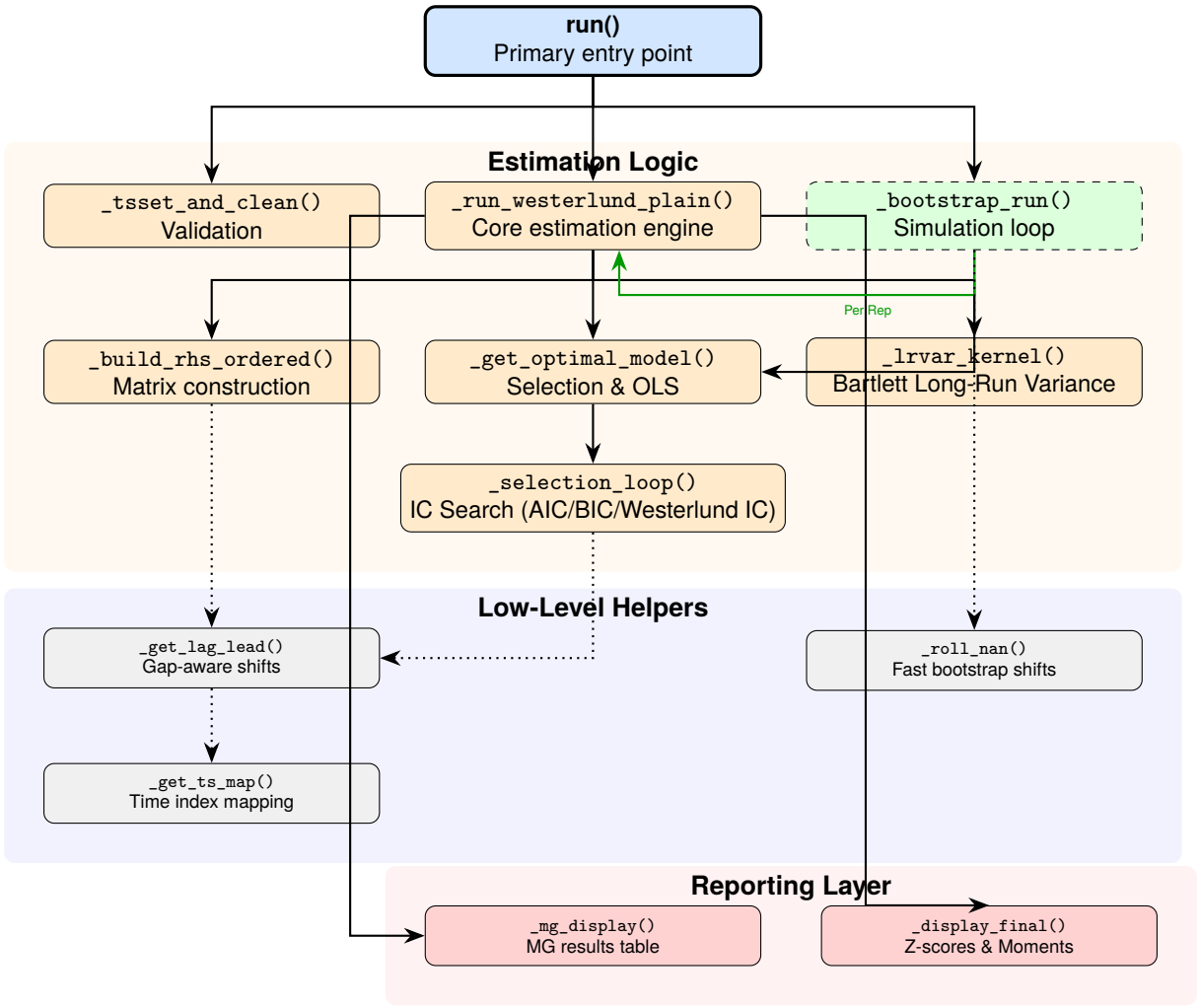

Figure 4: *Structure of the Python Version.*

Similarly to the R version, the Python method has a built-in gap-aware logic that does not rely on simple position-based shifting. The methods `_get_ts_map` and `_get_lag_lead` form the core of the gap-aware logic. The class maps values to their specific time index T. When a lag k is requested, the engine looks for the value at $T-k$. If the time index is discontinuous, the engine assigns a `NA` to that observation to prevent the model from erroneously calculating short-run dynamics across missing time periods.

4.1 Econometric Estimation Logic

4.1.1 Information Criterion and Model Selection

The `_selection_loop` identifies the optimal lag (p) and lead (q) lengths. For the specialized Westerlund criterion, the penalty function is defined the same way in R explained in Equation 7.

4.1.2 Long-Run Variance Estimation

The method `_lrvar_kernel` implements a Bartlett kernel (HAC) estimator which scales the same way as in the R version explained in Equation 8.

4.1.3 The Error Correction Model (ECM)

The `_get_optimal_model` method then fits the unrestricted ECM regression outlined in Equation 1.

4.1.4 Panel Statistics Construction

The `_run_westerlund_plain` method executes two estimation passes and estimates the group mean statistics and panel statistics the same way as outlined in Equations 9 and 11.

4.2 Bootstrap Inference

The `_bootstrap_run` method accounts for cross-sectional dependence through a recursive sieve bootstrap conducted under the null hypothesis ($H_0 : \alpha_i = 0$). The procedure follows the following steps:

1. **Null-Model Estimation:** For each unit i, the ECM is estimated with the null of no cointegration imposed via `_get_optimal_model(null_model=True)`. Residuals ($\hat{e}_{it}$) and regressor differences (ΔX_{it}) are extracted. To ensure the bootstrap innovations have zero mean, the residuals are centered using the mean of valid (non-NaN) indices, and regressor differences are centered using `np.nanmean`. The short-run autoregressive coefficients ($\hat{\phi}_{ij}$) and dynamic regressor coefficients ($\hat{\gamma}_{ij}$) are stored to drive the subsequent simulation.

2. **Synchronized Stable Shuffle:** To maintain the contemporaneous correlation structure $E[e_{it}e_{jt}] \neq 0$, a cluster bootstrap is performed on the time dimension. A set of indices U is drawn with replacement and duplicated to create an expanded pool. A single vector of random draws $U \sim (0, 1)$ is generated for the entire panel. A stable sort (`kind="mergesort"`) is applied to this key to generate a master permutation, ensuring that every unit in the panel is subjected to the exact same temporal realignment.

3. **Innovation Construction (u^*_{it}):** Composite bootstrap innovations are generated by combining the resampled residuals e^*_{it} with the dynamics of the resampled regressor differences, as outlined in Equation 13. The code implements a strict `NaN` propagation logic: if any component (the residual or the rolled regressor difference) is `NaN` due to boundary shifts, the sum u^*_{it} becomes `NaN`. To enforce finite-sample constraints, the first p observations are explicitly set to `NaN`. Only immediately before the recursion are these values converted to zero using `np.nan_to_num`.

4. **Recursive Simulation and Integration:** The bootstrap series are reconstructed in a two-stage process. First, the first-difference series Δy_{it}^* is generated recursively the same way outlined earlier in Equation 14. Second, the series are integrated to levels via cumulative summation (`np.cumsum`) the same way outlined earlier in Equation 15. To match the behavior of the original test, a boolean mask is applied to the levels y^* and X^*. This mask sets values to `NaN` for the initial p periods and any observations exceeding the original unit length $T_i + p$.

5. **Re-Estimation:** The levels are collected into a new panel DataFrame, dropping any rows where the masked dependent variable is `NaN`. The full Westerlund test is then executed on this simulated panel. This process is repeated for B replications to generate the empirical distribution of the G_t, G_a, P_t, and P_a statistics.

4.3 Standardization and Display

Standardized Z-scores are calculated in `_display_final` using the same way as explained in Equation 12.

4.4 Visualization

Similarly to its R counterpart, the Python implementation also provides a visualization tool, `plot_bootstrap`. It employs the `gaussian_kde` from `scipy.stats` to estimate the density, and then uses `ax.fill_between` to color the area under the kernel density estimation (KDE) curve. This visually represents where the statistics would fall if no cointegration existed.

Figure 5 provides an example of the visualization result generated by replicating the First Table in p.240 in Persyn and Westerlund (2008).

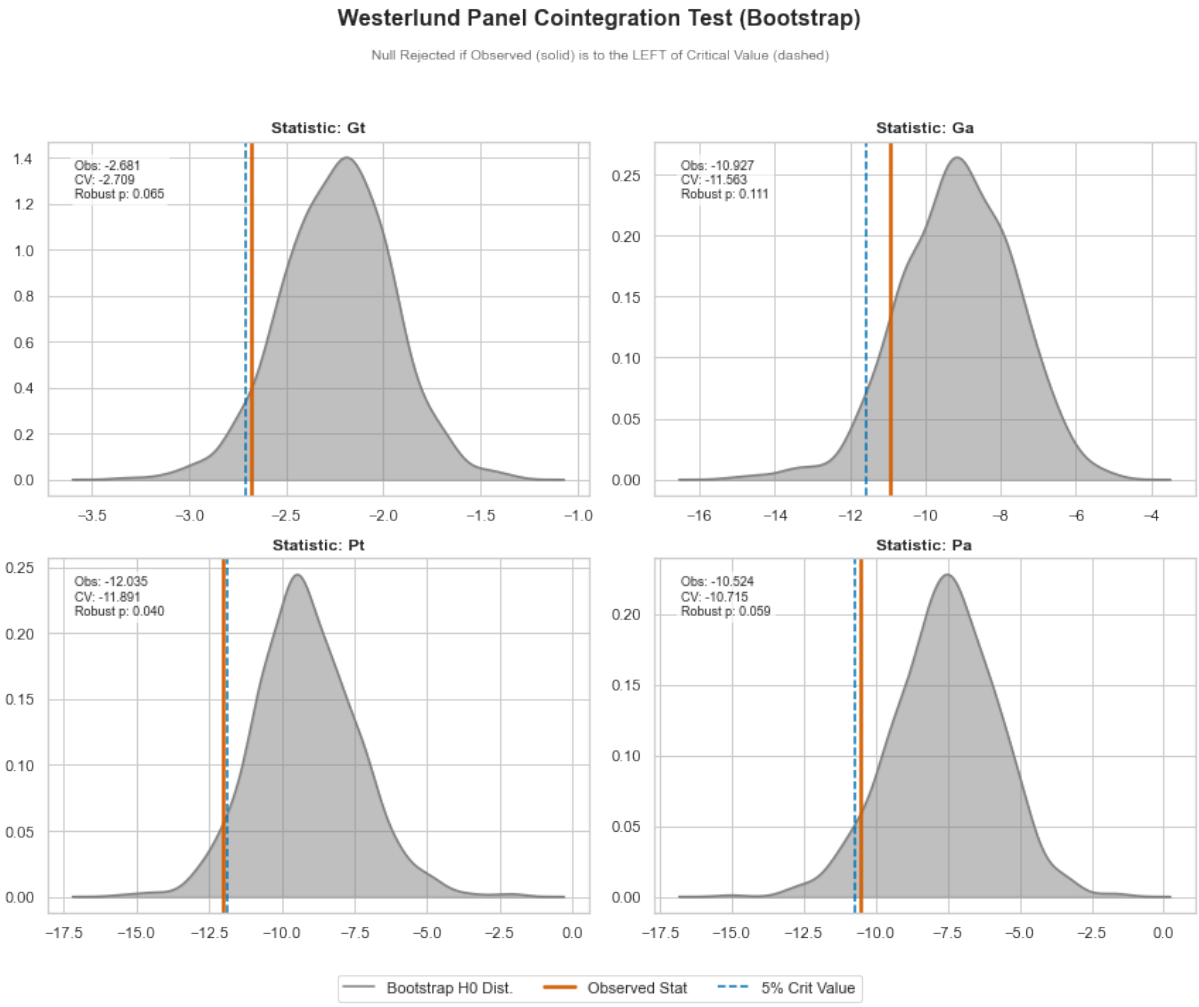

Figure 5: *Bootstrapping Distribution: Python Replication of the First Table in p.240 in Persyn and Westerlund (2008).*

5 Points to Note

There remains some implementation-level differences that would lead to minor discrepancies between the results given by the original `xtwest` function on Stata designed by Persyn and Westerlund (2008), and the Python and R versions outlined in this paper. If you run the asymptotic test (no bootstrap), the results should be nearly identical (up to floating-point precision). If you run the bootstrap, the robust p-values will vary slightly. One should be careful about the marginally significant cases if the bootstrapping times B is low. I provide some discussion of the known primary sources of differences and the implemented mitigation below.

5.1 Randomization

The Random Number Generation (RNG) mechanisms in these languages, including Stata, are fundamentally different and difficult to align. RNG is used when cross-sectional dependence is handled during

the bootstrap "shuffling" phase. The R implementation uses the standard `Mersenne-Twister` algorithm via `RNGkind`. On the other hand, the Python implementation uses the `numpy.random.Generator` with the `MT19937` core. Even with the same seed, the bit-stream of a Mersenne-Twister generator differs between R and Python's NumPy, meaning the resampled residuals (u^*_{it}) will not be identical and the resulting robust p-values will be slightly different.

5.2 OLS Equations

The two languages use different underlying linear algebra libraries for solving the OLS equations in the unit-specific ECMs. R (stats::lm) uses QR decomposition, as it is highly stable and handles near-collinear regressors by pivoting. On the other hand, Python (statsmodels.OLS) by default uses the Moore-Penrose pseudoinverse (`pinv`) to solve the least squares problem. I manually set it to use QR to align the results with R better.

5.3 Floating Point Drifts

Since the levels are built via integration and recursion, tiny differences are magnified.

6 Implementation Examples

In the following, I provide the results of the Python and R implementation replicating the results recorded by Persyn and Westerlund (2008). In the asymptotic cases, the results are almost perfectly similar, subject to display differences. In the robust cases, there are some minor discrepancies. Tables 4, 5, and 6 provide the results given by R, and Figure 2 gives the corresponding bootstrapping visualization. Tables 1, 2, and 3 provide the results given by Python, and Figure 5 gives the corresponding bootstrapping visualization. Seed 123 is used in both cases.

In the R cases, I call:

```
R> library("Westerlund")
R> df_westerlund_test <- haven::read_dta(
+   "/Users/boscohung/Documents/Westerlund/Stata/xtwestdata.dta"
+   )
R> table_7 <- westerlund_test(
+   data = df_westerlund_test,
+   yvar = "loghex",
+   xvars = c("loggdp"),
+   idvar = "ctr",
+   timevar = "year",
+   constant = TRUE,
+   lags = c(1, 3),
+   leads = c(0, 3),
+   lrwindow = 3,
+   trend = TRUE,
+   westerlund = TRUE
+ )
R> set.seed(123)
R> table_p240 <- westerlund_test(
+   data = df_westerlund_test,
+   yvar = "loghex",
+   xvars = c("loggdp"),
+   idvar = "ctr",
+   timevar = "year",
+   constant = TRUE,
+   lags = 1,
```

```
+   leads = 1,
+   bootstrap = 800,
+   lrwindow = 3,
+   trend = TRUE,
+   westerlund = FALSE
+ )
R> table_240_2 <- westerlund_test(
+   data = df_westerlund_test,
+   yvar = "loghex",
+   xvars = c("loggdp"),
+   idvar = "ctr",
+   timevar = "year",
+   constant = TRUE,
+   lags = c(1, 2),
+   leads = c(0, 1),
+   lrwindow = 2,
+   trend = TRUE
+ )
R> plot_westerlund_bootstrap(table_p240)
```

In the Python cases, I call:

```
py> from Westerlund import WesterlundTest
py> import pandas as pd
py> df_xtwest = pd.read_stata(
...     '/Users/boscohung/Documents/Westerlund/Stata/xtwestdata.dta'
... )
py> table_7 = WesterlundTest(
...     df_xtwest, "loghex", ["loggdp"], "ctr", "year",
...     lags=[1, 3], leads=[0, 3], lrwindow=3,
...     constant=True, trend=True, westerlund=True
... )
py> table_7_results = table_7.run()
py> table_240 = WesterlundTest(
...     df_xtwest, "loghex", ["loggdp"], "ctr", "year",
...     lags=1, leads=1, lrwindow=3, constant=True,
```

Table 1: *Python Replication of Table 7 in Westerlund (2007).*

With 20 series and 1 covariate
Average AIC selected lag length: 2.80
Average AIC selected lead length: 1.65

Statistic	Value	Z-value	*p*-value
Gt	−4.082	−9.613	0.000
Ga	−27.702	−10.626	0.000
Pt	−12.969	−4.100	0.000
Pa	−22.470	−10.119	0.000

Table 2: *Python Replication of the First Table in p.240 in Persyn and Westerlund (2008).*

Westerlund ECM Panel Cointegration Tests
Series: loghex ~ loggdp
N (Groups): 20
Lags: 1-1 | Leads: 1-1 | Window: 3

Statistic	Value	Z-score	*p*-value	Robust P
Gt	−2.681	−1.734	0.041	0.065
Ga	−10.927	0.714	0.762	0.111
Pt	−12.035	−2.961	0.002	0.040
Pa	−10.524	−1.160	0.123	0.059

Table 3: *Python Replication of the second table in p.240 in Persyn and Westerlund (2008).*

Westerlund ECM Panel Cointegration Tests
Series: loghex ~ loggdp
N (Groups): 20
Lags: 1-2 | Leads: 0-1 | Window: 2

Statistic	Value	Z-score	*p*-value	Robust P
Gt	−2.736	−2.036	0.021	–
Ga	−11.253	0.499	0.691	–
Pt	−12.859	−3.903	0.000	–
Pa	−11.773	−2.072	0.019	–

```
...     trend=True, bootstrap=800, seed=123
... )
py> table_240_results = table_240.run()
py> table_240.plot_bootstrap()
py> table_240_2 = WesterlundTest(
...     df_xtwest, "loghex", ["loggdp"], "ctr", "year",
...     lags=[1, 2], leads=[0, 1], lrwindow=2,
...     constant=True, trend=True
... )
py> table_240_2.run()
```

Table 4: *R Replication of Table 7 in Westerlund (2007).*
With 20 series and 1 covariate
Average AIC selected lag length: 2.80
Average AIC selected lead length: 1.65

Statistic	Value	Z-value	*p*-value
Gt	−4.082	−9.613	0.000
Ga	−27.702	−10.626	0.000
Pt	−12.969	−4.100	0.000
Pa	−22.470	−10.119	0.000

Table 5: *R Replication of the First Table in p.240 in Persyn and Westerlund (2008).*

Westerlund ECM Panel Cointegration Tests
Series: loghex ~ loggdp
N (Groups): 20
Lags: 1-1 | Leads: 1-1 | Window: 3

Statistic	Value	Z-score	*p*-value	Robust P
Gt	−2.681	−1.731	0.042	0.055
Ga	−10.927	0.713	0.762	0.100
Pt	−12.035	−2.959	0.002	0.029
Pa	−10.524	−1.160	0.123	0.054

Table 6: *R Replication of the second table in p.240 in Persyn and Westerlund (2008)*

Westerlund ECM Panel Cointegration Tests
Series: loghex ~ loggdp
N (Groups): 20
Lags: 1-2 | Leads: 0-1 | Window: 2

Statistic	Value	Z-score	*p*-value	Robust P
Gt	−2.736	−2.033	0.021	–
Ga	−11.253	0.499	0.691	–
Pt	−12.859	−3.902	0.000	–
Pa	−11.773	−2.072	0.019	–

7 Conclusion

The **Westerlund** package offers a convenient alternative for users without access to Stata or prefer using Python or R to run the Westerlund test to test cointegration. It offers an intuitive implementation and also functional approximation of the original **xtwest** function, while also introducing a visualization tool that enhances its utility.

Meanwhile, the current **Westerlund** implementation faces the same limitation of the **xtwest** function that they can only address cases with at most six regressors due to their reliance on a hard-coded table. Future work could explore allowing users to simulate the moment to enhance the precision of estimations and allow the use of more regressors. However, as the Westerlund test has a high parameter density which makes it power-hungry and macro panel data often suffers from a small N problem, use cases involving more than six regressors are probably less likely. Other pathways of future work would be to provide users with options to choose whether to apply finite sample correction, estimate the runtime, etc.

8 Bibliography

References

Croissant, Y., Millo, G., & Tappe, K. (2006, June). Plm: Linear Models for Panel Data [Institution: Comprehensive R Archive Network Pages: 2.6-7]. https://doi.org/10.32614/CRAN.package.plm

Hadri, K. (2000). Testing for stationarity in heterogeneous panel data. *The Econometrics Journal*, *3*(2), 148–161. https://doi.org/10.1111/1368-423X.00043

Im, K. S., Pesaran, M., & Shin, Y. (2003). Testing for unit roots in heterogeneous panels. *Journal of Econometrics*, *115*(1), 53–74. https://doi.org/10.1016/S0304-4076(03)00092-7

Levin, A., Lin, C.-F., & James Chu, C.-S. (2002). Unit root tests in panel data: Asymptotic and finite-sample properties. *Journal of Econometrics*, *108*(1), 1–24. https://doi.org/10.1016/S0304-4076(01)00098-7

Natsiopoulos, K., & Tzeremes, N. (2020, April). ARDL: ARDL, ECM and Bounds-Test for Cointegration [Institution: Comprehensive R Archive Network Pages: 0.2.4]. https://doi.org/10.32614/CRAN.package.ARDL

Newey, W. K., & West, K. D. (1994). Automatic Lag Selection in Covariance Matrix Estimation. *The Review of Economic Studies*, *61*(4), 631–653. https://doi.org/10.2307/2297912

Pedroni, P. (2004). Panel cointegration: Asymptotic and finite sample properties of pooled time series tests with an application to the ppp hypothesis. *Econometric Theory*, *20*(03). https://doi.org/10.1017/S0266466604203073

Persyn, D., & Westerlund, J. (2008). Error-Correction–Based Cointegration Tests for Panel Data. *The Stata Journal: Promoting communications on statistics and Stata*, *8*(2), 232–241. https://doi.org/10.1177/1536867X0800800205

Pesaran, M. H. (2007). A simple panel unit root test in the presence of cross-section dependence. *Journal of Applied Econometrics*, *22*(2), 265–312. https://doi.org/10.1002/jae.951

Pesaran, M. H. (2021). General diagnostic tests for cross-sectional dependence in panels. *Empirical Economics*, *60*(1), 13–50. https://doi.org/10.1007/s00181-020-01875-7

Pesaran, M. H., & Shin, Y. (1999). An Autoregressive Distributed-Lag Modelling Approach to Cointegration Analysis. In S. Strom (Ed.), *Econometrics and Economic Theory in the 20th Century* (pp. 371–413). Cambridge University Press. https://doi.org/10.1017/CCOL521633230.011

Pesaran, M. H., Shin, Y., & Smith, R. J. (2001). Bounds testing approaches to the analysis of level relationships. *Journal of Applied Econometrics*, *16*(3), 289–326. https://doi.org/10.1002/jae.616

Westerlund, J. (2007). Testing for Error Correction in Panel Data*. *Oxford Bulletin of Economics and Statistics*, *69*(6), 709–748. https://doi.org/10.1111/j.1468-0084.2007.00477.x

Zientara, P., & Kujawski, L. (2017, December). PooledMeanGroup: Pooled Mean Group Estimation of Dynamic Heterogenous Panels [Institution: Comprehensive R Archive Network Pages: 1.0]. https://doi.org/10.32614/CRAN.package.PooledMeanGroup